

Multimodal Deep Learning

Exercise Sheet 4 (Week 5)

Date: 27th of May, 2025

Exercise 1: Groups

For each set S and binary operation $*$ below, determine whether $(S, *)$ forms a group. If yes, verify all four group axioms. If no, identify which axiom fails with a counterexample.

(a) $(\mathbb{N}_0, +)$, where $\mathbb{N}_0 = \{0, 1, 2, \dots\}$.

(b) $(R_S, \circ)$, with

$$R_S = \{R_0, R_{90}, R_{180}, R_{270}\}, \quad R_\theta = \text{rotation of the square by } \theta^\circ \ (0 \leq \theta < 360).$$

(c) $(\{R_\theta \mid \theta \in \mathbb{R}\}, \times)$, where

$$R_\theta = \begin{pmatrix} \cos \theta & -\sin \theta \\ \sin \theta & \cos \theta \end{pmatrix},$$

and the operation is matrix multiplication.

Exercise 2: Rotational Symmetries in 2D

Let $\mathcal{G} = C_4$ be the group of 90° rotations in the two-dimensional plane, acting on square images of size $H \times W$ (assume $H = W$ for simplicity). Denote by g_{90} the element representing a clockwise rotation by 90° . The action of $g \in C_4$ on an image $\mathbf{I}$ is written $g \cdot \mathbf{I}$.

- Suppose you have a filter (e.g. a Sobel filter) that detects horizontal edges. Let $f(\mathbf{I})$ be the feature map obtained by applying this filter to an image $\mathbf{I}$. If you apply the filter to a 90° -rotated image, $f(g_{90} \cdot \mathbf{I})$, how would you expect this result to relate to $g_{90} \cdot f(\mathbf{I})$ (i.e. the original feature map rotated by 90°)? Is this an example of equivariance?
- A standard convolutional neural network (CNN) layer is typically composed of a convolution with learned filters, a non-linear activation (e.g. ReLU), and possibly a pooling operation. Is such a CNN layer (with filters learned from data without explicit symmetry constraints) generally equivariant to C_4 rotations? Explain why or why not.
- You are designing a neural network to analyse cell-microscopy images in which cell orientation is arbitrary and diagnostically irrelevant. Why would C_4 (or continuous $SO(2)$) equivariance or invariance be a desirable property for your network? Which parts of the network would you aim to make equivariant, and which parts invariant?

Exercise 3: Equivariance and Invariance

- Consider the task of object detection in images. An image $\mathbf{x} \in \mathbb{R}^{H \times W \times C}$ is input, and the output is a set of bounding boxes. If the image is translated, how should the bounding boxes ideally transform? Does this describe equivariance or invariance with respect to the translation group? Explain.

- (b) Consider the task of image classification. An image $\mathbf{x} \in \mathbb{R}^{H \times W \times C}$ is input, and the output is a class label (e.g., "cat" or "dog"). If the image is slightly rotated, should the class label change? Does this ideally describe equivariance or invariance with respect to a rotation group?

Exercise 4: Coding Exercise: Rotation Equivariance and Group Convolutions

Solve the exercises in the accompanying file "exercise4.ipynb".

Exercise 5: Applications

You are designing a CNN for detecting lung nodules in chest X-rays. The nodules can appear at any orientation.

- (a) Explain why rotation equivariance would be beneficial for this task.
- (b) Compare three approaches:
- Standard CNN with data augmentation
 - Group-equivariant CNN with C_4 symmetry
 - Steerable CNN with $SO(2)$ symmetry

Discuss the trade-offs in terms of:

- Parameter efficiency
 - Computational cost
 - Expected performance
- (c) Which approach would you recommend and why? Consider that medical datasets are typically small.

Exercise 6: Translation Equivariance

Let $T_{\mathbf{a}}$ denote translation by vector $\mathbf{a} \in \mathbb{R}^2$, and let $*$ denote the convolution operation. Assume zero-padding at the boundaries.

- (a) Prove that standard 2D convolution is translation-equivariant:

$$T_{\mathbf{a}}(f * k) = (T_{\mathbf{a}}f) * k$$

- (b) Given a 1D signal $x = [1, 2, 3, 4]$ and kernel $k = [1, -1]$, compute:

The convolution $y = x * k$

The translated signal $x' = T_1(x)$ (shift by 1)

Verify that $T_1(y) = x' * k$